Sisypush

1. 초기 확인

strings로 확인하면 플래그는 평문으로 드러나지 않는다.

```
Sisypush | height %um | scars %u | arrows or A/D push, Space strain, R reset
The stone remembers.
The stone rests.
```

IDA에서 핵심 루틴은 다음과 같다.

```
sub_140002913  main game loop
sub_1400011A0  checkpoint 생성
sub_140002570  매 프레임 물리/입력/높이 갱신
sub_140001AA0  scar 갱신 루틴
sub_140001440  flag prefix 복구
sub_1400015E0  flag suffix 1바이트 VM 복호화
sub_140002270  렌더링
```

2. main 흐름

sub_140002913에서 게임 상태가 스택에 초기화된다.

초기 상태 일부:

```
state+0x0c = 0x527d8a1f
state+0x10 = 4d 4a 53 45 43 20 24 07 10 13 37 88 48 5a 79 70
state+0x28 = 0x243f6a88
state+0x30 = 0x6a09e667f3bcc909
state+0x48 = 0x07102024
state+0x50 = encrypted tail blob
```

state+0x10 앞부분이 ASCII로 "MJSEC"이므로, 플래그 관련 상태라고 의심할 수 있다.

그 다음 실행 시점의 랜덤값을 섞어 state+0x38에 seed를 만든다.

```
seed = SDL_GetPerformanceCounter()
     ^ (SDL_GetTicks64() << 17)
```

```
      ^ 0x5359534951618A0E;

seed = splitmix64(seed);
```

이후 sub_1400011A0(&state)를 호출해 첫 hidden checkpoint를 생성한다.

3. 성공 분기

sub_140002913의 0x140002B83 근처에서 높이 조건이 나온다.

```
cmp eax, 2290h
```

0x2290 == 8848이다.

하지만 높이만 8848을 찍으면 전체 플래그가 아니라 prefix만 나온다. 전체 플래그 조건은 바로 아래에 있다.

```
cmp dword ptr [rbp-54h], 0Dh
cmp eax, 7F5F4734h
cmp eax, 714B5C73h
cmp rax, 9351AB846DBEBF66h
```

의미는 다음과 같다.

```
height >= 8848
scars == 13
state+0x0c == 0x7f5f4734
state+0x28 == 0x714b5c73
splitmix64(((uint64)state+0x0c << 32) ^ state+0x28 ^ state+0x30)
  == 0x9351ab846dbebf66
```

조건을 만족하면 "The stone remembers." 메시지 박스로 전체 플래그를 출력한다.

조건이 부족하면 "The stone rests."와 함께 prefix만 출력된다.

4. Hidden Checkpoint

sub_140002570을 보면 매 프레임 공의 위치와 높이를 갱신한다.

중요한 부분은 state+0x6f이다.

```
state+0x40 : checkpoint height
state+0x44 : checkpoint x
state+0x6f : checkpoint_seen
```

sub_140002570 안에서 다음 조건을 만족하면 state+0x6f = 1이 된다.

```
현재 이동 경로가 checkpoint_x를 지나감
현재 높이가 checkpoint height 이상
scars < 13
checkpoint_seen == 0
```

즉 checkpoint는 화면에는 보이지 않지만, 정상 플레이에서는 지나갈 수밖에 없는 지점으로 선택된다.

5. Scar 갱신

공이 다시 바닥으로 돌아오면 sub_140002570에서 sub_140001AA0이 호출된다.

sub_140001AA0 초반 조건:

```
cmp dword ptr [rcx+4Ch], 0Dh
cmp byte ptr [rcx+6Fh], 0
cmp dword ptr [rcx+24h], [rcx+40h]
```

의미:

```
scars < 13
checkpoint_seen == 1
climb_peak_m >= checkpoint_m
```

이 조건을 만족해야 scar가 1 증가하고 내부 상태가 섞인다.

따라서 풀이자는 다음 흐름을 이해해야 한다.

```
checkpoint 통과
목표 높이 이상 도달
바닥으로 복귀
scar 증가
```

```
다음 checkpoint 생성
총 13회 반복
```

6. 목표 높이 테이블

.rdata에 다음 13개 값이 있다.

```
0x02c6, 0x07e8, 0x0967, 0x0fe7,
0x02c6, 0x07e8, 0x096a, 0x1bbc,
0x07e8, 0x0197, 0x0967, 0x02c6,
0x2290
```

10진수:

```
710, 2024, 2407, 4071,
710, 2024, 2410, 7100,
2024, 407, 2407, 710,
8848
```

13번의 scar 갱신을 에뮬레이션하면 다음 상태가 된다.

```
state+0x0c = 0x7f5f4734
state+0x28 = 0x714b5c73

state+0x10 =
2a bc a2 87 f9 af 41 3f c0 97 81 f4 ef 31 16 e2

state+0x50 =
81 19 7e b8 89 dc b0 22 0b 52 23 09 bb 58 99 bd
0c f3 f3 de 32 16 78 df 41 66 7a 72 6e
```

7. state+0x30 역산

state+0x30은 직접 비교하지 않고 splitmix64 결과로 검증한다.

검증식:

```
x = (0x7f5f4734 << 32) ^ 0x714b5c73 ^ q30

z = x + 0x9e3779b97f4a7c15
z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9
z = (z ^ (z >> 27)) * 0x94d049bb133111eb
```

```
z = z ^ (z >> 31)

z == 0x9351ab846dbebf66
```

이 연산은 역산 가능하다. 역산하면:

```
state+0x30 = 0x91f7c2ab4d2e19f5
```

8. Prefix 복구

성공/실패 양쪽에서 sub_140001440이 호출된다.

```
call sub_140001440
```

인자로 height 8848이 들어가며, 결과는 다음 28바이트다.

```
MJSEC2026{5isyphus_1s_proud_
```

그래서 높이만 8848을 찍으면 이 앞부분만 볼 수 있다.

9. Suffix 복구

전체 조건을 만족하면 sub_140002913의 0x140002C80 근처 루프가 29번 돈다.

각 루프에서 sub_1400015E0을 호출한다.

```
call sub_1400015E0
```

호출 전 만들어지는 값은 다음 형태다.

```
mix =
    (state[0x10 + (i & 0xf)] << 24)
  ^ (i * 0x7f4a7c15 + 0xf612c926)
  ^ high32(state+0x30)
  ^ state+0x0c
  ^ low32(state+0x30)
  ^ state+0x28
```

```
lane = state[0x10 + ((i + 5) & 0xf)]
```

sub_1400015E0은 작은 VM처럼 동작하며 state+0x50에서 1바이트씩 suffix를 복호화한다.

29바이트 결과:

```
Of_you_f0r_getting_th1s_don3}
```

10. 최종 조합

Prefix:

```
MJSEC2026{5isyphus_1s_proud_
```

Suffix:

```
Of_you_f0r_getting_th1s_don3}
```

최종 플래그:

```
MJSEC2026{5isyphus_1s_proud_Of_you_f0r_getting_th1s_don3}
```